Universitatea Națională de Știință și Tehnologie
Politehnica București
Facultatea de Științe Aplicate
2026

# *Proiect Programare Liniară*

# *Optimizarea alocării serverelor într-un data center*

Facultatea de Științe Aplicate
Anul III, grupa 1333, semigr. B
Cercetări Operaționale - Proiect

Studenţi:

Stroescu Adrian-Gabriel

Tînjală Bianca-Ionela

Vlad Adrian

Volosenco Andreea-Laura

# Cuprins

**1. Istoria și Importanța Programării Liniare: Introducere în P.L.....4**
1.1. Contextul apariției Cercetărilor Operaționale și rolul managementului..... 4
1.2. Modelarea matematică a deciziei de alocare a serverelor.....5
1.3. Tipologia modelelor: Forma Canonică (FC) și Forma Standard (PLS)..... 6
**2. Metode de rezolvare: algoritmul simplex și metoda grafică.....8**
2.1. Metoda Grafică: Reprezentarea geometrică a regiunii admisibile.....8
2.1.1. Concluzii privind aplicabilitatea Metodei Grafice..... 10
2.2. Algoritmul Simplex: Principii de bază și explorarea vârfurilor.....10
2.2.1. Conceptul de Algoritm Simplex Primal (ASP)..... 10
2.2.2. Explorarea vârfurilor și baza soluției.....11
2.2.3. Logica tranziției între soluții.....11
2.3. Pașii Algoritmului Simplex: Testul de optimalitate și Pivotarea.....11
2.3.1. Criterii de selecție și Testul de Optimalitate.....11
2.3.2. Identificarea Elementului Pivot..... 12
3.3.3. Tehnica de Pivotare (Gauss-Jordan)..... 12
**3. Implementarea Computațională a Modelului.....13**
3.1. Implementarea algoritmului..... 13
3.2. Codul sursă.....13
3.2.1. Algoritmul de transformare PLS(transformare_PLS.py):.....13
3.2.2. Algoritmul pentru generarea tabelului Simplex(tabel_Simplex.py).....15
3.3. Algoritmul in PSEUDOCOD.....17
**4. Studiu de Caz: Optimizarea Alocării într-un Data Center.....19**
4.1. Formularea numerică a problemei de alocare a resurselor.....19
4.2. Rezolvarea iterativă prin tehnica tabelului Simplex..... 19
4.3. Interpretarea rezultatelor..... 23
**5. Concluzii.....24**
**6. Bibliografie:.....25**

# 1. Istoria și Importanța Programării Liniare: Introducere în P.L.

---

## 1.1. Contextul apariției Cercetărilor Operaționale și rolul managementului

Cercetarea Operațională (C.O.) este definită ca utilizarea metodelor științifice și a modelării matematice pentru a fundamenta deciziile în cadrul unor sisteme complexe. Deși rădăcinile sale teoretice sunt mai vechi, disciplina s-a cristalizat sub presiunea celui de-al Doilea Război Mondial, dintr-o necesitate practică de supraviețuire. În acel context, gestionarea resurselor extrem de limitate a forțat colaborarea dintre savanți și strategi, marcând câteva puncte de cotitură pentru managementul modern:

- Tranziția de la intuiție la calcul: Succesul misiunilor nu mai depindea doar de experiența liderilor, ci de optimizarea matematică a logisticii și a resurselor.
- Abordarea interdisciplinară: Pentru prima dată, matematicieni și ingineri au lucrat direct cu decidenții pentru a rezolva probleme operaționale în timp real.
- Transferul tehnologic postbelic: După 1945, aceste metode au migrat rapid spre sectorul civil, demonstrând că eficiența poate fi „calculată” și în fabrici, nu doar pe front.

Un moment crucial în această evoluție a fost anul 1947, când George Dantzig a formulat algoritmul Simplex. Acest instrument a transformat Programarea Liniară dintr-un concept abstract într-o soluție aplicabilă, punând bazele „managementului dezvoltat”. Astfel, managementul științific a încetat să mai caute simple soluții funcționale, orientându-se către identificarea soluției optime capabile să maximizeze performanța sub constrângeri stricte.

În peisajul actual, rolul managerului s-a redefinit complet. Acesta a devenit un arhitect de procese care folosește Cercetarea Operațională pentru a naviga prin complexitatea pieței. Programarea Liniară oferă astăzi structura necesară pentru a transforma variabilele de business în avantaje strategice, aducând claritate și predictibilitate într-un mediu economic aflat în continuă schimbare.

1.2. Modelarea matematică a deciziei de alocare a serverelor

Modelarea matematică a procesului de alocare a serverelor reprezintă etapa de transpunere a unei probleme decizionale reale într-un limbaj cantitativ riguros. În cadrul unei unități economice, principala problemă decizională este optimizarea producției prin folosirea eficientă a resurselor disponibile, fie ele materiale sau umane.

Conform structurii fundamentale a unei probleme de programare matematică, modelul este compus din următoarele elemente de bază:

$$Ip : f, g_i : X \subset \mathbb{R}^n \to \mathbb{R}$$

$$\begin{cases} (1) \quad \mathrm{opt}_X(f(x)), \quad x = (x_1, x_2, \ldots, x_n) \in X \\ (2) \quad g_i(x) \begin{pmatrix} \geq \\ \leq \\ = \end{pmatrix} b_i, \quad i = \overline{1, m}, \quad m \in \mathbb{N}^* \\ (3) \quad (x_1, x_2, \ldots, x_k) \in X_1 \subset X, \quad k \leq n \end{cases}$$

$$C : \mathrm{opt}_X(f(x)) = ?$$

Semnificația elementelor în contextul serverelor:

Optimul (opt): Reprezintă atingerea valorii maxime (max sau sup) sau minime (min sau inf).

Funcția f(x): Funcția obiectiv (scop sau de eficiență) care măsoară performanța (ex: beneficiu maxim sau cost minim de operare).

↳ dacă f(x) reprezintă un beneficiu (profit) - se urmărește maximizarea (opt=MAX).

↳ dacă f(x) reprezintă un cost (cheltuială, pierdere) - se urmărește minimizarea (opt=MIN).

Restricțiile $g_i(x)$: Condițiile impuse variabilelor în raport cu resursele $b_i$ se numesc restricțiile problemei.

Variabilele $(x_1,x_2,....,x_n)$: Sunt variabilele problemei, iar mulțimea reprezintă spațiul soluțiilor problemei.

Cantitățile $b_i = \overline{1, m}$ reprezintă resursele disponibile.

În programarea liniară, atât funcția obiectiv $f(x)$, cât și funcțiile $g_i(x)$ sunt funcții liniare. Modelul este completat de condițiile de nenegativitate ($x_j \geq 0$), reflectând faptul că alocarea resurselor hardware nu poate avea valori negative. Prin această structură, managementul serverelor devine o știință a deciziei, transformând variabilele tehnice într-un plan de operare optim.

1.3. Tipologia modelelor: Forma Canonică (FC) și Forma Standard (PLS)

În studiul programării liniare, după formularea modelului general, este necesară încadrarea acestuia într-o tipologie specifică pentru a putea aplica algoritmii de rezolvare.

## A. Forma Canonică (FC)

O problemă se află în formă canonică dacă toate restricțiile de bază sunt inegalități, iar sensul acestora este determinat de natura funcției obiectiv prin relații de concordanță sau discordanță.

1. Pentru problema de MAXIM (opt = MAX):

Relația de Concordanță (RC): Restricțiile trebuie să fie de tipul "mai mic sau egal" (≤).

Relația de Discordanță (RD): Restricțiile de tipul "mai mare sau egal" (≥) sunt considerate în discordanță.

Modelul FC pentru MAX presupune ca toate restricțiile să fie aduse la forma de concordanță (≤).

2. Pentru problema de MINIM (opt = MIN):

Relația de Concordanță (RC): Restricțiile trebuie să fie de tipul "mai mare sau egal" (≥).

Relația de Discordanță (RD): Restricțiile de tipul "mai mic sau egal" (≤) sunt considerate în disconcordanță.

Modelul FC pentru MIN presupune ca toate restricțiile să fie aduse la forma de concordanță (≥).

Condiția obligatorie: În ambele cazuri, toate variabilele trebuie să fie nenegative ($x_j \geq 0$).

- **Forma generală**:

$$opt\ f(x) = \sum_{j=1}^{n} c_j \cdot x_j = c_1 \cdot x_1 + c_2 \cdot x_2 + \cdots + c_n \cdot x_n$$

$$\begin{cases} a_{11} \cdot x_1 + a_{12} \cdot x_2 + \cdots + a_{1n} \cdot x_n \\ a_{21} \cdot x_1 + a_{22} \cdot x_2 + \cdots + a_{2n} \cdot x_n \\ \vdots \\ a_{m1} \cdot x_1 + a_{m2} \cdot x_2 + \cdots + a_{mn} \cdot x_n \end{cases} \begin{pmatrix} \leq \\ \geq \\ = \end{pmatrix} \begin{matrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{matrix}$$

$$x_1, x_2, \ldots, x_n \begin{pmatrix} \leq 0 \\ \geq 0 \\ \in \mathbb{R} \end{pmatrix}$$

Aceasta poate fi scrisă și sub forma matriceală astfel:

$$\text{opt}\, f(x) = C \cdot x$$
$$A \cdot x \begin{pmatrix} \leq \\ \geq \\ = \end{pmatrix} b$$
$$x \begin{matrix} \leq 0 \\ \geq 0 \\ \in \mathbb{R} \end{matrix}$$

unde:

$opt = min/max$ - tipul problemei

$C = (c_1, c_2, \dots, c_n)$ - reprezintă coeficienții funcției obiectiv

$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$ - reprezintă variabilele problemei;

$b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$ - reprezintă resursele disponibile de producție

$A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix} \in \mathcal{M}_{m \times n}(\mathbb{R})$ -reprezintă matricea coeficienților tehnologici

Forma canonică scrisă în forma matriceala pentru problema de MAX:

$$\max f(x) = C \cdot x$$
$$A \cdot x \leq b$$
$$x \geq 0$$

Forma canonică scrisă în forma matriceală pentru problema de MIN:

$$\min f(x) = C \cdot x$$
$$A \cdot x \geq b$$
$$x \geq 0$$

## B. Forma Standard (PLS)

Forma Standard este modelul în care toate restricțiile (cu excepția celor de nenegativitate) sunt de tip egalitate (=). Această formă este obligatorie pentru utilizarea algoritmului Simplex.

Elementele PLS:

- Funcția obiectiv: $opt\ f(x) = \sum_{j=1}^{n} c_j \cdot x_j = c_1 \cdot x_1 + c_2 \cdot x_2 + \cdots + c_n \cdot x_n$

- Restricții: Ax = b (sistem de ecuații)
- Nenegativitate: x ≥ 0

### C. Reguli de standardizare (Trecerea la PLS)

Pentru transformarea unui model oarecare în Forma Standard, se aplică următoarele reguli:

Regula R1 (Transformarea variabilelor) - se aplică condițiilor 3 din PL:

- Dacă o variabilă este nepozitivă (x≤0) - se înlocuiește cu x = -x', unde x' ≥ 0.
- Dacă o variabilă este liberă (x aparține lui R) - se înlocuiește cu diferența a două variabile nenegative: x = x' - x", unde x', x" ≥ 0.

Regula R2 (Transformarea restricțiilor) - se aplică restricțiilor 2 din PL

- Pentru inegalități de tip "≤": Se adaugă o variabilă de compensare (ecart), notată cu y ≥ 0, pentru a transforma inegalitatea în egalitate.
- Pentru inegalități de tip " ≥": Se scade o variabilă de compensare (ecart) y ≥ 0 și se adaugă o variabilă de penalizare (auxiliară), notată cu z ≥ 0.
- În funcția obiectiv: Variabilele de penalizare (z) se introduc cu coeficientul +M pentru problemele de MIN și -M pentru problemele de MAX (unde M este un număr foarte mare, tinzând spre infinit).

## 2. Metode de rezolvare: algoritmul simplex și metoda grafică

---

2.1. Metoda Grafică: Reprezentarea geometrică a regiunii admisibile

Metoda grafică se aplică la rezolvarea problemelor de programare liniară cu două variabile de decizie, $x_1$ și $x_2$. Această metodă furnizează o interpretare vizuală a conceptelor de mulțime convexă și de puncte extreme.

### Etapele rezolvării grafice:

1. Construirea sistemului de axe și reprezentarea restricțiilor

Fiecare restricție de tip inegalitate din cadrul modelului:

$$a_{i1}x_1 + a_{i2}x_2 \leq b_i$$

este tratată inițial ca o dreaptă de ecuație $a_{i1}x_1 + a_{i2}x_2 = b_i$. Această dreaptă împarte planul $R^2$ în două semiplane. Se determină semiplanul care satisface inegalitatea (de regulă, verificând dacă originea $O(0,0)$ aparține acestuia).

**2. Determinarea regiunii admisibile (Poligonul soluțiilor)**

Regiunea admisibilă $S$ este intersecția tuturor semiplanelor determinate de restricțiile tehnice și de restricțiile de nenegativitate $x_1 \geq 0, x_2 \geq 0$.

Dacă intersecția este vidă, problema nu are soluții.

Dacă intersecția este o suprafață închisă, aceasta formează un poligon convex.

**3. Reprezentarea funcției obiectiv (Dreapta de izoprofit/izocost)**

Se alege o valoare arbitrară $k$ pentru funcția obiectiv $f(x) = c_1x_1 + c_2x_2$ și se trasează dreapta:

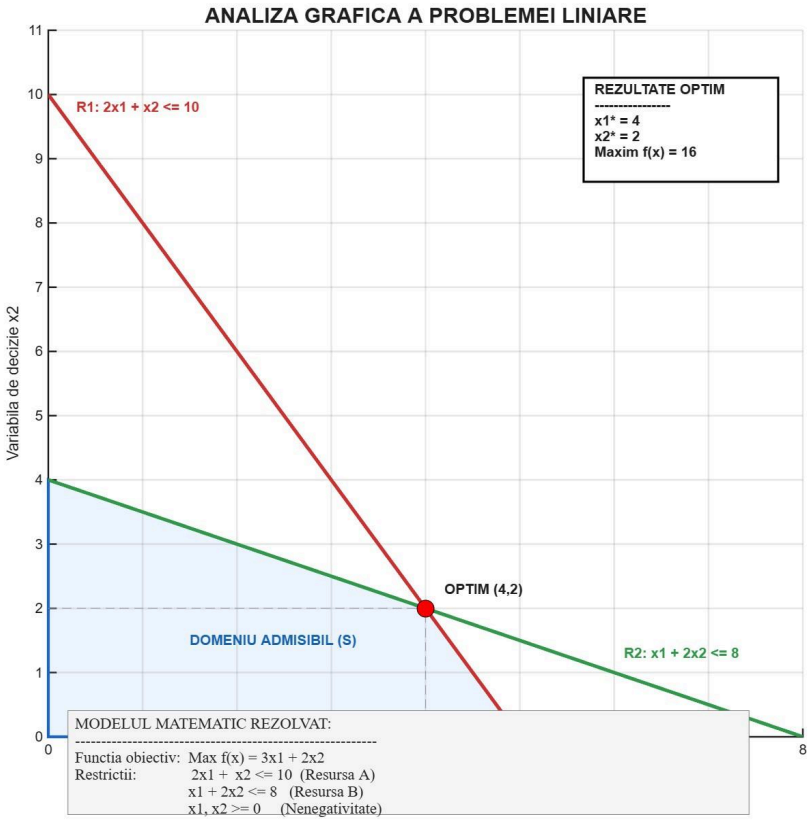
$$c_1x_1 + c_2x_2 = k$$

Vectorul numeric $c = (c_1, c_2)$ reprezintă gradientul funcției și indică direcția de creștere maximă a acesteia.

**4. Identificarea punctului optim**

Se deplasează dreapta funcției obiectiv paralel cu ea însăși în direcția vectorului gradient (pentru maximizare) sau în direcție opusă (pentru minimizare):

- Soluție unică: Ultimul punct (vârf) al poligonului $S$ pe care dreapta îl atinge înainte de a părăsi regiunea admisibilă reprezintă soluția optimă $x^*$.
- Soluții multiple: Dacă dreapta funcției obiectiv devine paralelă cu una dintre laturile poligonului, toate punctele de pe acel segment sunt soluții optime.
- Optim infinit: Dacă regiunea $S$ este nemărginită în direcția de optimizare, funcția poate crește/scădea spre infinit.



### 2.1.1. Concluzii privind aplicabilitatea Metodei Grafice

Rezultatul obținut prin reprezentarea geometrică nu este unul accidental, ci reprezintă transpunerea vizuală a fundamentelor teoretice ale programării liniare, sintetizate prin două teoreme esențiale:

- Legătura cu Teorema 1 (Teorema Convexității): Această teoremă garantează că regiunea admisibilă $S$, formată prin intersecția semiplanelor restricțiilor, va fi întotdeauna o mulțime convexă. Vizual, acest lucru înseamnă că poligonul soluțiilor nu are scobituri sau întreruperi, permițând funcției obiectiv să „alunece” continuu pe suprafața sa fără a întâlni obstacole logice.
- Legătura cu Teorema 2 (Teorema Punctelor Extreme): Această teoremă justifică eficiența metodei prin faptul că soluția optimă se află obligatoriu într-un vârf. În timp ce interiorul poligonului conține o infinitate de puncte (soluții posibile), Teorema 2 ne demonstrează că este suficient să evaluăm funcția obiectiv $f(x) = c^T x$ doar în punctele de intersecție ale dreptelor (vârfurile poligonului), deoarece acolo se ating valorile de maxim sau minim.

În esență, metoda grafică confirmă vizual ceea ce Teorema 1 și Teorema 2 stabilesc analitic: optimizarea liniară este procesul de identificare a celui mai favorabil punct extrem dintr-o structură geometrică riguros definită și convexă.

## 2.2. Algoritmul Simplex: Principii de bază și explorarea vârfurilor

Algoritmul Simplex, fundamentat de George Dantzig, reprezintă metoda de referință în programarea liniară pentru determinarea soluției optime. Acesta nu analizează toate punctele din interiorul regiunii admisibile $S$, ci se bazează pe rezultatele fundamentale ale Teoremelor 1 și 2, care garantează existența optimului într-un punct extrem.

### 2.2.1. Conceptul de Algoritm Simplex Primal (ASP)

Algoritmul Simplex Primal (ASP) este procedura care operează direct asupra variabilelor din problema originală, căutând să îmbunătățească valoarea funcției obiectiv menținând în permanență admisibilitatea soluției.

**Principiile de bază ale ASP în explorarea bazei sunt:**

- Admisibilitatea Primală: Toate soluțiile temporare identificate pe parcursul iterațiilor trebuie să respecte restricțiile de nenegativitate ($x_j \geq 0$).
- Deplasarea pe frontiere: ASP navighează exclusiv pe muchiile poliedrului convex, trecând de la o Soluție de Bază Admisibilă (SBA) la alta adiacentă.
- Îmbunătățirea Monotonă: La fiecare iterație, algoritmul selectează un vârf vecin care asigură o valoare a funcției obiectiv cel puțin egală cu cea curentă, evitând astfel reîntoarcerea la soluții inferioare.

2.2.2. Explorarea vârfurilor și baza soluției

Din punct de vedere algebric, un vârf al poliedrului corespunde unei baze de calcul. Algoritmul explorează aceste vârfuri prin modificarea compoziției bazei:

Variabilele de bază ($x_B$): Sunt variabilele care pot lua valori strict pozitive și definesc poziția curentă pe frontiera domeniului.

Variabilele secundare ($x_N$): Sunt variabilele fixate la valoarea zero. Din punct de vedere geometric, acestea reprezintă restricțiile care sunt "active" (pe care se află vârful curent).

2.2.3. Logica tranziției între soluții

În cadrul ASP, explorarea nu este aleatorie. Algoritmul utilizează informația furnizată de gradientul funcției obiectiv pentru a determina care dintre muchiile ce pleacă din vârful curent conduce cel mai rapid către optim. Această explorare se oprește atunci când nu mai există nicio muchie adiacentă care să permită creșterea (sau scăderea, în cazul minimizării) valorii funcției obiectiv.

2.3. Pașii Algoritmului Simplex: Testul de optimalitate și Pivotarea

Algoritmul Simplex Primal ($ASP$) reprezintă motorul computațional al programării liniare, fiind o procedură iterativă care pleacă de la o Soluție de Bază Admisibilă (SBA). Acesta transformă iterativ o soluție în alta, îmbunătățind succesiv valoarea funcției obiectiv până la atingerea punctului optim.

2.3.1. Criterii de selecție și Testul de Optimalitate

Orice iterație începe cu evaluarea soluției curente prin calculul indicatorilor de optim $\Delta_j$. Aceștia se determină prin formula:

$$\Delta_j = c_j - z_j \text{ , unde } z_j = \sum_{i \in I} c_i A_{ij} .$$

Regulile de decizie pentru modificarea bazei sunt determinate de tipul optimului:

| Operație | Maximizare (maxf) | Minimizare (minf) |
|---|---|---|
| Condiție de Optim | Toți $\Delta_j \leq 0$ | Toți $\Delta_j \geq 0$ |
| Variabila care intră ($a_q$) | $\Delta_q = \max\{\Delta_j > 0\}$ | $\Delta_q = \min\{\Delta_j < 0\}$ |
| Variabila care iese ($a_p$) | Raportul $p = \min\{\frac{x_{Bi}}{A_{iq}} \mid A_{iq} > 0\}$ | Raportul $p = \min\{\frac{x_{Bi}}{A_{iq}} \mid A_{iq} > 0\}$ |

2.3.2. Identificarea Elementului Pivot

Elementul aflat la intersecția liniei p (variabila care iese) cu coloana q (variabila care intră) se numește pivot.

Coloana Pivot (q): Identificată prin testul de optim.

Linia Pivot (p): Identificată prin raportul minim p.

Validitate: Pivotul trebuie să fie întotdeauna strict pozitiv ($A_{pq} > 0$). Dacă pe coloana q toți $A_{iq} \leq 0$, problema are optim infinit.

3.3.3. Tehnica de Pivotare (Gauss-Jordan)

Recalcularea tabelului pentru noua iterație se realizează prin metoda eliminării complete.

A. Normalizarea liniei pivot (p): Elementele noii linii se obțin prin împărțirea liniei vechi la pivot:

$$\hat{A}_{pj} = \frac{A_{pj}}{A_{pq}}$$

B. Recalcularea restului tabelului (Regula dreptunghiului): Toate celelalte elemente se calculează folosind diferența dintre produsul elementelor de pe diagonala principală și cel de pe diagonala secundară, raportată la pivot:

$$\widehat{A}_{ij} = \frac{\text{Diag. Principală} - \text{Diag. Secundară}}{\text{Pivot}} = \frac{(A_{ij} \cdot A_{pq}) - (A_{iq} \cdot A_{pj})}{A_{pq}}$$

Unde:

$A_{ij} \cdot A_{pq}$ (Diagonala Principală): Produsul dintre elementul vechi de calculat și pivot.

$A_{iq} \cdot A_{pj}$ (Diagonala Secundară): Produsul dintre elementul de pe coloana pivotului și cel de pe linia pivotului, corespunzătoare poziției elementului $A_{ij}$.

În noul tabel, coloana variabilei intrate devine vector unitar (1 în poziția pivotului și 0 în rest).

# 3. Implementarea Computațională a Modelului

## 3.1. Implementarea algoritmului

Algoritmul pentru rezolvarea Problemei de Programare Liniară a fost implementat în limbajul de programare Python (versiunea 3.13), utilizând următoarele biblioteci și module:

- customtkinter (versiunea 5.2.2)
- darkdetect (versiunea 0.8.0)
- numpy (versiunea 2.4.3)
- packaging (versiunea 26.0)

Aplicația respectă principiile programării orientate pe obiecte, fiind structurată pe clase și module separate pentru interfață, logică și validări.

Complexitatea algoritmului implementat este de ordinul O(n · m), unde n reprezintă numărul de restricții, iar m numărul de variabile, datorită operațiilor matriciale din cadrul metodei Simplex(ASP).

## 3.2. Codul sursă

### 3.2.1. Algoritmul de transformare PLS(transformare_PLS.py):

```
def transformarePLS(c, a, b, semne, cond_semn, tip_problema):
    # Creeaza matricea restrictiilor si lista coeficientilor functiei obiectiv
    ma, mc = np.array(a, dtype=float), list(c)
    # Numele variabilelor originale (x1, x2, ...)
    nv_orig = [f"x{i + 1}" for i in range(len(c))]

    # Liste pentru matricea finala, costuri si nume variabile
    af, cf, nf = [], [], []

    # Tratam variabilele in functie de conditiile de semn
    for j in range(len(c)):
        reg = cond_semn[j].get()  # Conditia de semn pentru variabila

        # Daca variabila <= 0
        if "≤ 0" in reg:
            af.append(-ma[:, j])            # Inmulteste coloana cu -1
            cf.append(-mc[j])               # Inmulteste coeficientul cu -1
            nf.append(f"{nv_orig[j]}'")     # Marcheaza variabila

        # Daca variabila este libera (∈ R)
        elif "∈ R" in reg:
            af.append(ma[:, j])   # Partea pozitiva
            cf.append(mc[j])
            nf.append(f"{nv_orig[j]}+")

            af.append(-ma[:, j])  # Partea negativa
            cf.append(-mc[j])
            nf.append(f"{nv_orig[j]}-")

        # Daca variabila >= 0 (caz standard)
        else:
            af.append(ma[:, j])
            cf.append(mc[j])
```

```
        nf.append(nv_orig[j])

# Constanta M folosita pentru variabile artificiale
constanta_M = 10000 if tip_problema == "MIN" else -10000
# Vectorul termenilor liberi
bs = np.array(b, dtype=float)

# Liste pentru variabile slack si artificiale
slack_cols, slack_costs, slack_names = [], [], []
art_cols, art_costs, art_names = [], [], []

# Configuratia bazei initiale
baza_config, cy, cz = [], 1, 1

# Parcurge fiecare restrictie
for i in range(len(bs)):
    s = semne[i].get()  # Tipul restrictiei

    # Restrictie de tip <=
    if s == "≤":
        col = np.zeros(len(bs))
        col[i] = 1  # Adauga variabila slack
        slack_cols.append(col)
        slack_costs.append(0.0)
        slack_names.append(f"y{cy}")

        baza_config.append(('y', len(slack_names) - 1))
        cy += 1

    # Restrictie de tip >=
    elif s == "≥":
        csur, cart = np.zeros(len(bs)), np.zeros(len(bs))
        csur[i], cart[i] = -1, 1  # Variabila surplus si artificiala

        slack_cols.append(csur)
        slack_costs.append(0.0)
        slack_names.append(f"y{cy}")

        art_cols.append(cart)
        art_costs.append(constanta_M)
        art_names.append(f"z{cz}")

        baza_config.append(('z', len(art_names) - 1))
        cy += 1
        cz += 1

    # Restrictie de tip =
    else:
        cart = np.zeros(len(bs))
        cart[i] = 1  # Variabila artificiala

        art_cols.append(cart)
        art_costs.append(constanta_M)
        art_names.append(f"z{cz}")

        baza_config.append(('z', len(art_names) - 1))
        cz += 1

# Construieste matricea finala (concatenare coloane)
ms = np.column_stack(af + slack_cols + art_cols)

# Vectorul costurilor
cs = cf + slack_costs + art_costs

# Lista numelor variabilelor
nf_final = nf + slack_names + art_names

# Determina indicii bazei initiale
```

```
    bz = []
    for tip_v, r_i in baza_config:
        if tip_v == 'y':
            bz.append(len(nf) + r_i)
        else:
            bz.append(len(nf) + len(slack_names) + r_i)

    # Returneaza matricea, costurile, termenii liberi, baza si numele variabilelor
    return ms, np.array(cs), bs, bz, nf_final
```

### 3.2.2. Algoritmul pentru generarea tabelului Simplex(tabel_Simplex.py)

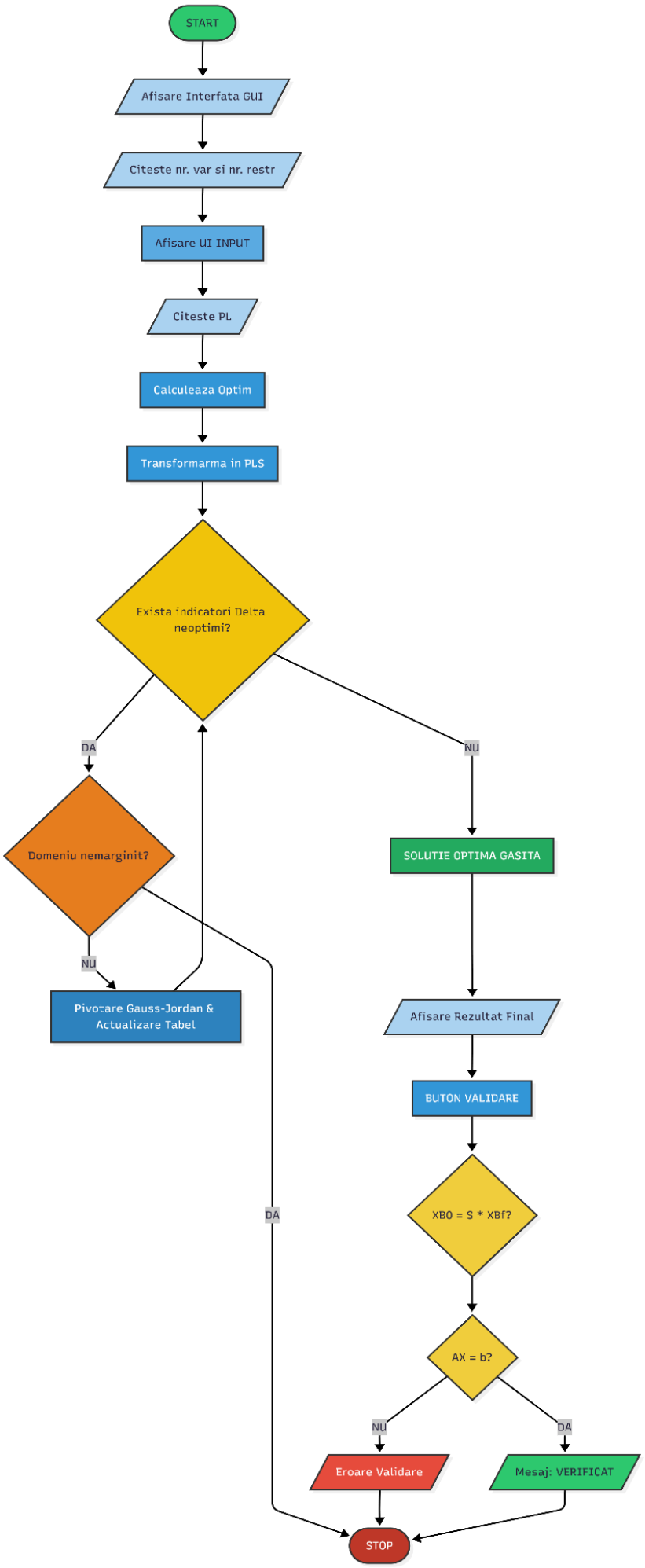
```
class tabelSimplex(ctk.CTkFrame):
    def __init__(self, container, titlu, matrice, coloana_xb, costuri, deltas, baza,
nume_variabile, pivot=None):
        super().__init__(container, border_width=2, border_color="#9bcfff",
fg_color="#ffffff")
        self.pack(pady=8, padx=10, fill="x")

        # titlul tabelului (ex: iteratia curenta sau solutie finala)
        ctk.CTkLabel(
            self,
            text=titlu,
            font=("Montserrat", 14, "bold"),
            text_color="#294280",
            fg_color="transparent"
        ).pack(pady=2)

        # zona unde desenam tabelul simplex
        f_grila = ctk.CTkFrame(self, fg_color="transparent")
        f_grila.pack(pady=5)

        nr_linii, nr_coloane = len(baza), len(costuri)

        # culori folosite in tabel (doar pentru evidentiere)
        CULOARE_CRUCE = "#9bcfff"
        CULOARE_PIVOT = "#5284ff"
        CULOARE_NEGATIV = "#FF5252"
        CULOARE_POZITIV = "#2ECC71"

        # afisam numele variabilelor sus in tabel
        for j in range(nr_coloane):
            ctk.CTkLabel(
                f_grila,
                text=nume_variabile[j],
                width=75,
                font=("Montserrat", 12, "bold"),
                text_color="#ffffff",
                fg_color="#6d91b3"
            ).grid(row=0, column=3 + j)

        # linia cu costurile Cj
        ctk.CTkLabel(f_grila, text="Cj", width=75, fg_color="#cde7ff").grid(row=1,
column=2)

        for j, val in enumerate(costuri):
            # daca suntem pe coloana pivot, o evidentiem
            fundal = CULOARE_CRUCE if (pivot and j == pivot[1]) else "#cde7ff"
            ctk.CTkLabel(f_grila, text=transformaraFractie(val), width=75,
fg_color=fundal).grid(row=1, column=3 + j)

        # anteturile coloanelor principale
        for j, text_antet in enumerate(["CB", "B", "XB"]):
            ctk.CTkLabel(
                f_grila,
                text=text_antet,
                width=75,
```

```
                font=("Montserrat", 12, "bold"),
                text_color="#ffffff",
                fg_color="#6d91b3"
            ).grid(row=2, column=j)

        # construim tabelul pe randuri
        for i in range(nr_linii):
            idx_variabila_baza = baza[i]

            # evidentiem linia pivot daca exista
            fundal_linie = CULOARE_CRUCE if (pivot and i == pivot[0]) else
"transparent"

            # CB, B si XB (partea din stanga a tabelului)
            ctk.CTkLabel(f_grila,
text=transformaraFractie(costuri[idx_variabila_baza]), width=75,
fg_color=fundal_linie).grid(row=3 + i, column=0)
            ctk.CTkLabel(f_grila, text=nume_variabile[idx_variabila_baza], width=75,
fg_color=fundal_linie).grid(row=3 + i, column=1)
            ctk.CTkLabel(f_grila, text=transformaraFractie(coloana_xb[i]), width=75,
fg_color=fundal_linie).grid(row=3 + i, column=2)

            # valorile din matricea simplex
            for j in range(nr_coloane):
                culoare_celula = "transparent"

                # daca avem pivot, coloram linia si coloana lui
                if pivot:
                    if i == pivot[0] and j == pivot[1]:
                        culoare_celula = CULOARE_PIVOT
                    elif i == pivot[0] or j == pivot[1]:
                        culoare_celula = CULOARE_CRUCE

                # font mai gros doar pe pivot
                if pivot and i == pivot[0] and j == pivot[1]:
                    font_celula = ("Montserrat", 12, "bold")
                else:
                    font_celula = ("Montserrat", 12)

                ctk.CTkLabel(
                    f_grila,
                    text=transformaraFractie(matrice[i, j]),
                    width=75,
                    fg_color=culoare_celula,
                    font=font_celula
                ).grid(row=3 + i, column=3 + j)
        ctk.CTkLabel(
            f_grila,
            text="Delta_j",
            font=("Montserrat", 12, "bold")
        ).grid(row=3 + nr_linii, column=2, pady=5)

        for j, d_val in enumerate(deltas):
            # coloram valorile in functie de semn
            culoare_text = CULOARE_POZITIV if d_val >= -0.0001 else CULOARE_NEGATIV

            # evidentiem coloana pivot daca exista
            fundal_delta = CULOARE_CRUCE if (pivot and j == pivot[1]) else
"transparent"

            ctk.CTkLabel(
                f_grila,
                text=transformaraFractie(d_val),
                font=("Montserrat", 12, "bold"),
                text_color=culoare_text,
                fg_color=fundal_delta,
                width=75
            ).grid(row=3 + nr_linii, column=3 + j)
```

## 3.3. Algoritmul in PSEUDOCOD

```
DATE DE INTRARE: tip_opt, n, m, C[], A[][], Semne[], B[]
DATE DE IESIRE: X_final[], f_optim, mesaj_validare

INCEPUT
   // Pasul 1: Pregatirea datelor (Standardizare)
   citeste tip_opt, n, m, C, A, Semne, B


   ┌ pentru fiecare restrictie i de la 1 la m executa
   │   ┌ daca Semne[i] = "≤" atunci
   │   │    adauga variabila slack y[i]
   │   │  altfel daca Semne[i] = "≥" atunci
   │   │    scade surplus y[i], adauga artificiala z[i]
   │   │  altfel daca Semne[i] = "=" atunci
   │   │    adauga artificiala z[i]
   │   └
   └
   salveaza coloane_initiale in DictColoane

   // Pasul 3: Ciclul iterativ Simplex
   ┌ repeta
   │   calculeaza Delta[j] pentru fiecare coloana j
   │   cp <- indicele coloanei cu cel mai bun Delta[j]
   │
   │   ┌ daca Delta[cp] este optim atunci
   │   │    iesi din repeta
   │   └
   │
   │   calculeaza rapoarte theta[i] = B[i] / A[i][cp] pentru A[i][cp] > 0
   │
   │   ┌ daca nu exista niciun raport valid atunci
   │   │    scrie "Domeniu nemarginit"
   │   │    STOP
   │   └
   │
   │   rp <- indicele liniei cu theta[i] minim
   │
   │   // Pivotare Gauss-Jordan
   │   normalizeaza linia rp prin impartire la pivotul A[rp][cp]
   │   pentru fiecare linie i != rp executa
   │      elimina elementul de pe coloana cp din linia i
   │   sfarsit_pentru
   │
   └ pana cand optimul este gasit

   // Pasul 4: Validarea rezultatelor
   ┌ Metoda_I (Transfer):
   │    verifica daca B_orig = S * XB_final (folosind DictColoane)
   │  Metoda_II (Fezabilitate):
   │    verifica daca A_orig * X_final respecta Semne[] fata de B_orig
   └

   scrie X_final, f_optim, mesaj_validare
SFARSIT
```

## 3.4. Schemă logică



# 4. Studiu de Caz: Optimizarea Alocării într-un Data Center

4.1. Formularea numerică a problemei de alocare a resurselor.

## Enunțul și Ipoteza Problemei

O companie de hosting administrează un Data Center și dorește să determine planul optim de furnizare a serviciilor pentru a maximiza profitul lunar. Compania oferă trei tipuri de configurații virtuale: Standard ($x_1$), Premium ($x_2$) și Dedicated ($x_3$).

Succesul operațiunii depinde de gestionarea a trei resurse critice:

Unități de procesare (CPU): necesare pentru rularea instrucțiunilor.

Memorie RAM: esențială pentru stabilitatea serviciilor.

Lățime de bandă (Traffic): necesară pentru transferul de date.

## Formularea Modelului Matematic (P.L.)

Modelul în Forma Generală (FG) este compus din funcția obiectiv și sistemul de restricții:

Funcția obiectiv (maximizarea profitului):
$\max f(x) = 40x_1 + 25x_2 + 20x_3$

Restricțiile de resurse (inegalități):

$$0,8x_1 + 0,4x_2 + 0,2x_3 \leq 16 \text{ (Limitare CPU)}$$

$$1x_1 + 2x_2 + 1x_3 \leq 20 \text{ (Limitare RAM)}$$

$$0,5x_1 + 0,5x_2 + 1x_3 \leq 15 \text{ (Limitare Traffic)}$$

Condiții de nenegativitate:
$x_1, x_2, x_3 \geq 0$

4.2. Rezolvarea iterativă prin tehnica tabelului Simplex

**Pas 1: (Standardizare P.L $\Rightarrow$ P.L.S)**

$\max f = 40x_1 + 25x_2 + 20x_3 + 0y_1 + 0y_2 + 0y_3$

Sistemul de ecuații:

$$0,8x_1 + 0,4x_2 + 0,2x_3 + y_1 = 16$$

$$1x_1 + 2x_2 + 1x_3 + y_2 = 20$$

$$0,5x_1 + 0,5x_2 + 1x_3 + y_3 = 15$$

Condiții: $x_{1,2,3} \geq 0$ ; $y_{1,2,3} \geq 0$

Unde: $x_1, x_2, x_3$ = variabile principale; $y_1, y_2, y_3$ = variabile secundare.

**Pas 2: (Tehnica Tabelului Simplex - T.S.)**

P.L.S $\Rightarrow n = 6, m = 3, c = (40, 25, 20, 0, 0, 0)$.

**T1 (Matricea A și vectorul b):**

| c1 | c2 | c3 | c4 | c5 | c6 | b |
|---|---|---|---|---|---|---|
| 0,8 | 0,4 | 0,2 | 1 | 0 | 0 | 16 |
| 1 | 2 | 1 | 0 | 1 | 0 | 20 |
| 0,5 | 0,5 | 1 | 0 | 0 | 1 | 15 |
| $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | |

T2 (Identificarea bazei inițiale B):

| | x1 | x2 | x3 | y1 | y2 | y3 |
|---|---|---|---|---|---|---|
| | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ |
| | 0,5 | 0,3 | 0 | 1 | 0 | 0 |
| | 1 | 1,5 | 0,5 | 0 | 1 | 0 |
| | 0,2 | 0,1 | 0,5 | 0 | 0 | 1 |

$$B = \{a_4, a_5, a_6\} \qquad C_B = (0, 0, 0) \qquad X_B = \begin{pmatrix} 12 \\ 18 \\ 10 \end{pmatrix}$$

**Rezolvare cu Tabel Simplex:**

| | MAX | | 40 | 25 | 20 | 0 | 0 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| $C_B$ | B | $X_B$ | $a_1 \downarrow$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | |
| 0 | $a_4$ | 16 | 0,8 | 0,4 | 0,2 | 1 | 0 | 0 | $\leftarrow C_B, B, X_B$ $A_{ij}$ |
| 0 | $a_5 \uparrow$ | 20 | [1] | 2 | 1 | 0 | 1 | 0 | $I_0$ |
| 0 | $a_6$ | 15 | 0,5 | 0,5 | 1 | 0 | 0 | 1 | |
| | | | 0 | 0 | 0 | 0 | 0 | 0 | $\rightarrow z_j$ |
| | $z^0 = 0$ | | $\Delta_1 = 40$ | $\Delta_2 = 25$ | $\Delta_3 = 20$ | $\Delta_4 = 0$ | $\Delta_5 = 0$ | $\Delta_6 = 0$ | $\rightarrow \Delta_j$ |
| 0 | $a_4$ | 0 | 0 | -1,2 | -0,6 | 1 | -0,8 | 0 | |
| 40 | $a_1$ | 20 | 1 | 2 | 1 | 0 | 1 | 0 | $I_1$ |
| 0 | $a_6$ | 5 | 0 | -0,5 | 0,5 | 0 | -0,5 | 1 | |
| | | | 40 | 80 | 40 | 0 | 40 | 0 | $\rightarrow z_j$ |
| | $z^1 = 800$ | | $\Delta_1 = 0$ | $\Delta_2 = -55$ | $\Delta_3 = -20$ | $\Delta_4 = 0$ | $\Delta_5 = -40$ | $\Delta_6 = 0$ | $\rightarrow \Delta_j = I_{STOP}$ |

**Detalierea Calculelor și Iterațiilor:**

1. Iterația $I_0$ (Baza inițială $y_1, y_2, y_3$
Coeficienții bazei sunt $C_B = [0, 0, 0]$.

Calculul valorilor $z_j$ (Costuri):

- $z_1 = (0 \cdot 0,8) + (0 \cdot 1) + (0 \cdot 0,5) = 0$
- $z_2 = (0 \cdot 0,4) + (0 \cdot 2) + (0 \cdot 0,5) = 0$

- $z_3 = (0 \cdot 0,2) + (0 \cdot 1) + (0 \cdot 1) = 0$
- $z_4, z_5, z_6 = 0$ (corespunzător matricei unitate)
  $Z^0 = (0 \cdot 16) + (0 \cdot 20) + (0 \cdot 15) = \mathbf{0}$

Calculul diferențelor $\Delta_j$ ($c_j - z_j$):

- $\Delta_1 = 40 - 0 = \mathbf{40}$
- $\Delta_2 = 25 - 0 = \mathbf{25}$
- $\Delta_3 = 20 - 0 = \mathbf{20}$
- $\Delta_4, \Delta_5, \Delta_6 = 0 - 0 = 0$

Identificarea Pivotului (P):

- CVB (Intră): $\max(\Delta_1, \Delta_2, \Delta_3) = 40 \Rightarrow$ coloana $a_1$.
- CAB (Iese): $\min\{\frac{X_{B_i}}{A_{i1}} > 0\} \Rightarrow \min\{\frac{16}{0,8}, \frac{20}{1}, \frac{15}{0,5}\} = \min\{20, 20, 30\}$..
- Alegem rândul al doilea (corespunzător $a_5$) conform tabelului
- Pivotul (P): Intersecția coloanei $a_1$ cu rândul $a_5 \Rightarrow \mathbf{P = 1}$.

2. Iterația $I_1$ (Baza optimă $y_1, x_1, y_3$)

Coeficienții bazei devin $C_B = [0, 40, 0]$.

Calculul valorilor $z_j$

- $z_1 = (0 \cdot 0) + (40 \cdot 1) + (0 \cdot 0) = \mathbf{40}$
- $z_2 = (0 \cdot -1,2) + (40 \cdot 2) + (0 \cdot -0,5) = \mathbf{80}$
- $z_3 = (0 \cdot -0,6) + (40 \cdot 1) + (0 \cdot 0,5) = \mathbf{40}$
- $z_4 = (0 \cdot 1) + (40 \cdot 0) + (0 \cdot 0) = \mathbf{0}$
- $z_5 = (0 \cdot -0,8) + (40 \cdot 1) + (0 \cdot -0,5) = \mathbf{40}$
- $z_6 = (0 \cdot 0) + (40 \cdot 0) + (0 \cdot 1) = \mathbf{0}$
- Funcția obiectiv:

  $$Z^1 = (0 \cdot 0) + (40 \cdot 20) + (0 \cdot 5) = \mathbf{800}$$

Calculul diferențelor $\Delta_j$ ($c_j - z_j$):

- $\Delta_1 = 40 - 40 = \mathbf{0}$
- $\Delta_2 = 25 - 80 = \mathbf{-55}$
- $\Delta_3 = 20 - 40 = \mathbf{-20}$
- $\Delta_4 = 0 - 0 = \mathbf{0}$
- $\Delta_5 = 0 - 40 = \mathbf{-40}$
- $\Delta_6 = 0 - 0 = \mathbf{0}$

Pivotul în $I_1$:

Deoarece toți $\Delta_j \leq 0$, testul de optimalitate este îndeplinit. Nu mai există pivot, algoritmul se oprește la $I_{STOP}$.

## Rezultate Finale:

**Soluția P.L.S ($I_{STOP}$):**

$y_1 = 0, x_1 = 20, y_3 = 5$.

$x_2 = 0, x_3 = 0, y_2 = 0$.

$z_{max} = 800$.

**Soluția P.L:** $x_{opt} = (20, 0, 0)$ **;** $F_{max} = 800$**.**

**Verificare:**

$\max f(x) = 40(20) + 25(0) + 20(0) = 800$ (A)

$b = S \cdot \overline{X}_B$: $\begin{pmatrix} 1 & 0,8 & 0 \\ 0 & 1 & 0 \\ 0 & 0,5 & 1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 20 \\ 5 \end{pmatrix} = \begin{pmatrix} 16 \\ 20 \\ 15 \end{pmatrix} = b$ (A).

4.3. Interpretarea rezultatelor

În urma rezolvării cu metoda Simplex, am ajuns la concluzia că pentru a scoate un profit maxim de 800 de unități, cel mai bine este să ne concentrăm doar pe serviciul Standard ($x_1$), producând 20 de unități. Celelalte două variante, Premium și Dedicated, nu intră în soluția optimă, ceea ce înseamnă că la prețurile și consumul actual, mai mult ne-ar încurca profitul decât să-l ajute.

Analiza utilizării resurselor indică faptul că memoria RAM și procesorul (CPU) sunt complet saturate, reprezentând principalii factori limitativi în maximizarea profitului. În prezent, aceste resurse înregistrează variabile de ecart zero, confirmând atingerea capacității hardware maxime. În schimb, lățimea de bandă (Traffic) prezintă un excedent de 5 unități. Recomandăm suplimentarea capacității de procesare și a memoriei RAM pentru a fructifica disponibilitatea rețelei. Până la aceste upgrade-uri, alocarea a 20 de unități Standard rămâne configurația optimă pentru eficiența Data Center-ului.

## 5. Concluzii

Analiza efectuată în cadrul acestei lucrări evidențiază eficiența programării liniare ca instrument indispensabil în procesul decizional modern. Prin utilizarea algoritmului Simplex, am demonstrat că optimizarea nu reprezintă doar o aproximare a succesului, ci o determinare matematică exactă a punctului în care resursele limitate generează randamentul maxim. Această abordare elimină riscurile asociate deciziilor empirice, oferind o metodologie riguroasă pentru gestionarea complexității în orice sistem de producție sau servicii.

Analiza critică a metodei:

Avantaje:

- Obiectivitate: Metoda furnizează o soluție optimă fundamentată pe date cuantificabile, asigurând o transparență totală a procesului decizional.
- Identificarea punctelor critice: Algoritmul permite localizarea precisă a resurselor deficitare, oferind managerilor indicatori clari privind necesarul de investiții viitoare.
- Versatilitate: Modelul poate fi extins și adaptat pentru sisteme cu un număr ridicat de variabile, fiind ideal pentru implementări software automatizate.

Limitări:

- Dependența de liniaritate: Modelul presupune relații constante între variabile, neputând reflecta fidel fenomene precum fluctuațiile de preț în funcție de volum sau uzura variabilă a echipamentelor.
- Natura statică: Soluția este valabilă doar pentru setul de date actual; orice modificare în mediul extern (piață, costuri, tehnologie) necesită o reluare integrală a calculului.

Notă personală:

Analiza utilizării resurselor indică faptul că memoria RAM și procesorul (CPU) sunt complet saturate, reprezentând principalii factori limitativi în maximizarea profitului. În prezent, aceste resurse înregistrează variabile de ecart zero, confirmând atingerea capacității hardware maxime. În schimb, lățimea de bandă (Traffic) prezintă un excedent de 5 unități. Recomandăm suplimentarea capacității de procesare și a memoriei RAM pentru a fructifica disponibilitatea rețelei. Până la aceste upgrade-uri, alocarea a 20 de unități Standard rămâne configurația optimă pentru eficiența Data Center-ului.

## 6. Bibliografie:

---


1. **Bibic Simona Mihaela**, *Cercetări Operaționale, suport de curs electronic, https://archive.curs.upb.ro/2024/course/view.php?id=9543.*
2. **Hillier, F.S. and Lieberman, G.J**., *Introduction to Operations Research*, McGraw-Hill Education.
3. **Gheorghe Boldur-Lățescu**, *Cercetări operaționale: Programare liniară*, Editura Didactică și Pedagogică, București.
4. **Bazaraa, M.S., Jarvis, J.J. and Sherali**, H.D., *Linear Programming and Network Flows*, John Wiley & Sons.
5. **Marchand, P., & Holland, T. S. (2016)**. *Graphics and GUIs with MATLAB. CRC Press.*
6. https://cloudconvert.com/png-to-ico
7. https://www.rapidtables.com/math/symbols/Basic_Math_Symbols.html
8. https://numpy.org
9. https://docs.python.org/3/library/fractions.html
10. https://mermaid.live
11. https://docs.python.org/3/
12. https://www.w3schools.com/python/default.asp
13. https://customtkinter.tomschimansky.com
14. https://pbinfo.ro/
15. A doua metoda de verificare este generata cu Gemini.